



Smart Contract Security Audit Report

Mellow Interop Audit

1. Contents

1.	Contents	2
2.	General Information	3
2.1.	Introduction	3
2.2.	Scope of Work	3
2.3.	Threat Model.....	4
2.4.	Weakness Scoring	4
2.5.	Disclaimer	5
3.	Summary.....	5
3.1.	Suggestions	5
4.	General Recommendations	6
4.1.	Security Process Improvement	6
5.	Findings.....	7
5.1.	Lack of oracle value validation	7
5.2.	Oracle value must be set before the pool is deployed.....	7
5.3.	Lack of value verification in setValue function	8
5.4.	sourceCoreAddress verified twice and sourceEndpointId not verified.....	9
5.5.	Incorrect _convertToAssets implementation	9
5.6.	Delegator Contract should use burn instead of clear.....	10
5.7.	Incompatibility with Fee-on-Transfer Tokens	11
6.	Appendix.....	12
6.1.	About us	12

2. General Information

This report contains information about the results of the security audit of the Mellow Protocol (hereafter referred to as “Customer”) smart contracts, conducted by [Decurity](#) in the period from 10/03/2025 to 14/03/2025.

2.1. Introduction

Tasks solved during the work are:

- Review the protocol design and the usage of 3rd party dependencies,
- Audit the contracts implementation,
- Develop the recommendations and suggestions to improve the security of the contracts.

2.2. Scope of Work

The audit scope included the contracts in the following repository: [mellow-interop](#). Initial review was done for the commit [57b45a](#). The re-test was done for commit [d75644](#).

The following contracts have been tested:

- `src/core/TargetCore.sol`
- `src/core/SourceCore.sol`
- `src/core/TargetCoreStorage.sol`
- `src/core/SourceCoreStorage.sol`
- `src/utls/Delegator.sol`
- `src/utls/WithdrawalQueue.sol`
- `src/utls/Oracle.sol`
- `src/oft/MellowOFT.sol`
- `src/oft/MellowOFTAdapter.sol`

2.3. Threat Model

The assessment presumes actions of an intruder who might have capabilities of any role (an external user, token owner, token service owner, a contract). The centralization risks have not been considered upon the request of the Customer.

The main possible threat actors are:

- User,
- Protocol owner,
- Liquidity Token owner/contract.

The table below contains sample attacks that malicious attackers might carry out.

Table. Theoretically possible attacks

Attack	Actor
Contract code or data hijacking <i>Deploying a malicious contract or submitting malicious data</i>	Contract owner Token owner
Financial fraud <i>A malicious manipulation of the business logic and balances, such as a re-entrancy attack or a flash loan attack</i>	Anyone
Attacks on implementation <i>Exploiting the weaknesses in the compiler or the runtime of the smart contracts</i>	Anyone

2.4. Weakness Scoring

An expert evaluation scores the findings in this report, an impact of each vulnerability is calculated based on its ease of exploitation (based on the industry practice and our experience) and severity (for the considered threats).

2.5. Disclaimer

Due to the intrinsic nature of the software and vulnerabilities and the changing threat landscape, it cannot be generally guaranteed that a certain security property of a program holds.

Therefore, this report is provided “as is” and is not a guarantee that the analyzed system does not contain any other security weaknesses or vulnerabilities. Furthermore, this report is not an endorsement of the Customer’s project, nor is it an investment advice.

That being said, Decurity exercises best effort to perform their contractual obligations and follow the industry methodologies to discover as many weaknesses as possible and maximize the audit coverage using the limited resources.

3. Summary

As a result of this work, we have discovered multiple low-level security issues. The Mellow team has given the feedback for the suggested changes and explanation for the underlying code.

3.1. Suggestions

The table below contains the discovered issues, their risk level, and their status as of March 19, 2025.

Table. Discovered weaknesses

Issue	Contract	Risk Level	Status
Lack of oracle value validation	src/utls/Oracle.sol	Low	Acknowledged
Oracle value must be set before the pool is deployed	src/core/SourceCore.sol	Low	Fixed
Lack of value verification in setValue function	src/utls/Oracle.sol	Low	Acknowledged

Issue	Contract	Risk Level	Status
sourceCoreAddress verified twice and sourceEndpointId not verified	src/core/TargetCoreStorage.sol	Low	Fixed
Incorrect _convertToAssets implementation	src/core/SourceCore.sol	Low	Fixed
Delegator Contract should use burn instead of clear	src/utls/Delegator.sol	Low	Acknowledged
Incompatibility with Fee-on-Transfer Tokens	src/core/SourceCore.sol	Info	Acknowledged

4. General Recommendations

This section contains general recommendations on how to improve overall security level.

The Findings section contains technical recommendations for each discovered issue.

4.1. Security Process Improvement

The following is a brief long-term action plan to mitigate further weaknesses and bring the product security to a higher level:

- Keep the whitepaper and documentation updated to make it consistent with the implementation and the intended use cases of the system,
- Perform regular audits for all the new contracts and updates,
- Ensure the secure off-chain storage and processing of the credentials (e.g. the privileged private keys),
- Launch a public bug bounty campaign for the contracts.

5. Findings

5.1. Lack of oracle value validation

Risk Level: Low

Status: Acknowledged

Contracts:

- src/Utils/Oracle.sol

Description:

When setting the oracle value using the `setValue()` function in the Oracle contract, no verification is performed to ensure that the calculated `totalAssets()` in the SourceCore contract do not exceed the actual asset balance held by the SourceCore and OFTAdapter

The SourceCore contract calculates the total assets as:

```
function totalAssets() public view override(IERC4626, ERC4626Upgradeable)
returns (uint256) {
    return Math.mulDiv(totalSupply(), oracle().getValue(), D18);
}
```

If the oracle value is set too high functions that depend on `totalAssets()`, such as deposit, mint, or withdrawal calculations, will operate under incorrect assumptions, potentially leading to over-minting of tokens or allowing withdrawals larger than the assets available.

Remediation:

Consider ensuring that `previewRedeem(totalSupply) <= balanceOf(SourceCore) + balanceOf(OFTAdapter)` when setting new oracle value.

5.2. Oracle value must be set before the pool is deployed

Risk Level: Low

Status: Fixed in the commit [d14ba9](#).

Contracts:

- src/core/SourceCore.sol

Description:

If the oracle value is not set when the SourceCore is deployed, the calculation of total assets becomes faulty.

```
function totalAssets() public view override(IERC4626, ERC4626Upgradeable)
returns (uint256) {
    return Math.mulDiv(totalSupply(), oracle().getValue(), D18);
}

function convertToShares(uint256 assets, Math.Rounding rounding) internal view
virtual returns (uint256) {
    return assets.mulDiv(totalSupply() + decimalsOffset(), totalAssets() + 1,
rounding);
}
```

As a result, even a small asset deposit will be converted into a disproportionately large number of shares. This flaw allows a malicious actor to mint an excessive amount of shares by exploiting the uninitialized or zero oracle value.

Remediation:

Consider setting the oracle value before deploying SourceCore.

5.3. Lack of value verification in `setValue` function

Risk Level: Low

Status: Acknowledged

Contracts:

- `src/utils/Oracle.sol`

Location: Function: `setValue()`.

Description:

The `setValue()` function currently does not include any value verification. This absence of validation could potentially result in inaccurate updates to the oracle's value.

Remediation:

To address this issue, a verification mechanism should be implemented within the `setValue()` function. This mechanism should ensure that any value being set is both greater than 0 and has the correct number of decimal places.

5.4. `sourceCoreAddress` verified twice and `sourceEndpointId` not verified

Risk Level: Low

Status: Fixed in the commit [d14ba9](#).

Contracts:

- `src/core/TargetCoreStorage.sol`

Location: Function: `__TargetCoreStorage_init()`.

Description:

The `__TargetCoreStorage_init()` function contains a duplicate check for the value of `params.sourceCoreAddress` being zero, resulting in different error messages. However, there is a lack of validation for `params.sourceEndpointId`, which should also be verified for completeness and consistency.

```
if (params.sourceCoreAddress == bytes32(0)) {  
    revert("TargetCore: zero value");  
}  
  
if (params.sourceCoreAddress == bytes32(0)) {  
    revert("TargetCore: zero address");  
}
```

Remediation:

To address this issue, the validation for `params.sourceEndpointId` should be implemented as follows:

```
if (params.sourceEndpointId == 0) {  
    revert("TargetCore: zero value");  
}  
  
if (params.sourceCoreAddress == bytes32(0)) {  
    revert("TargetCore: zero address");  
}
```

5.5. Incorrect `_convertToAssets` implementation

Risk Level: Low

Status: Fixed in the commit [d14ba9](#).

Contracts:

- `src/core/SourceCore.sol`

Location: Function: `pushToTarget()`.

Description:

During the execution of the `pushToTarget()` function, there is an attempt to convert the `pendingShares` into assets for the purpose of comparing them with the current `liquid` balance. This conversion process deviates from the standard implementation in ERC4626, potentially resulting in minor calculation inaccuracies.

Remediation:

To address this issue, it is recommended to utilize the original formula specified in OpenZeppelin's ERC4626 contract for the conversion process.

References:

- <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/ca7a4e39de0860bbaadf95824207886e6de9fa64/contracts/token/ERC20/extensions/ERC4626.sol#L233>

5.6. Delegator Contract should use burn instead of clear

Risk Level: Low

Status: Acknowledged

Contracts:

- `src/utis/Delegator.sol`

Description:

The Delegator contract currently utilizes a `clear()` function to handle cases where LayerZero is malfunctioning. However, LayerZero recommends using the `burn()` function instead, as it provides a more robust and reliable way to remove messages from the processing queue.

Remediation:

The LayerZero recommends using `burn()` function for this purpose.

The burn function operates similarly to the `clear` function with two key distinctions:

1. The OApp is not required to be aware of the original payload
2. The nonce designated for burning must be less than the lazyInboundNonce

References:

- <https://docs.layerzero.network/v2/developers/evm/troubleshooting/debugging-messages#nilify-and-burn>

5.7. Incompatibility with Fee-on-Transfer Tokens

Risk Level: Info**Status:** Acknowledged**Contracts:**

- `src/core/SourceCore.sol`

Description:

The OFT standard on LayerZero does not support fee-on-transfer tokens, because the logic built into the OFT smart contracts assumes that the actual amount of tokens transferred remains constant and corresponds exactly to the specified amount. The introduction of a fee deduction mechanism leads to discrepancies, which in turn can cause errors in token distribution and compromise the integrity of cross-chain operations.

Remediation:

Make sure that `asset()` is not a fee-on-transfer token.

6. Appendix

6.1. About us

The [Decurity](#) team consists of experienced hackers who have been doing application security assessments and penetration testing for over a decade.

During the recent years, we've gained expertise in the blockchain field and have conducted numerous audits for both centralized and decentralized projects: exchanges, protocols, and blockchain nodes.

Our efforts have helped to protect hundreds of millions of dollars and make web3 a safer place.